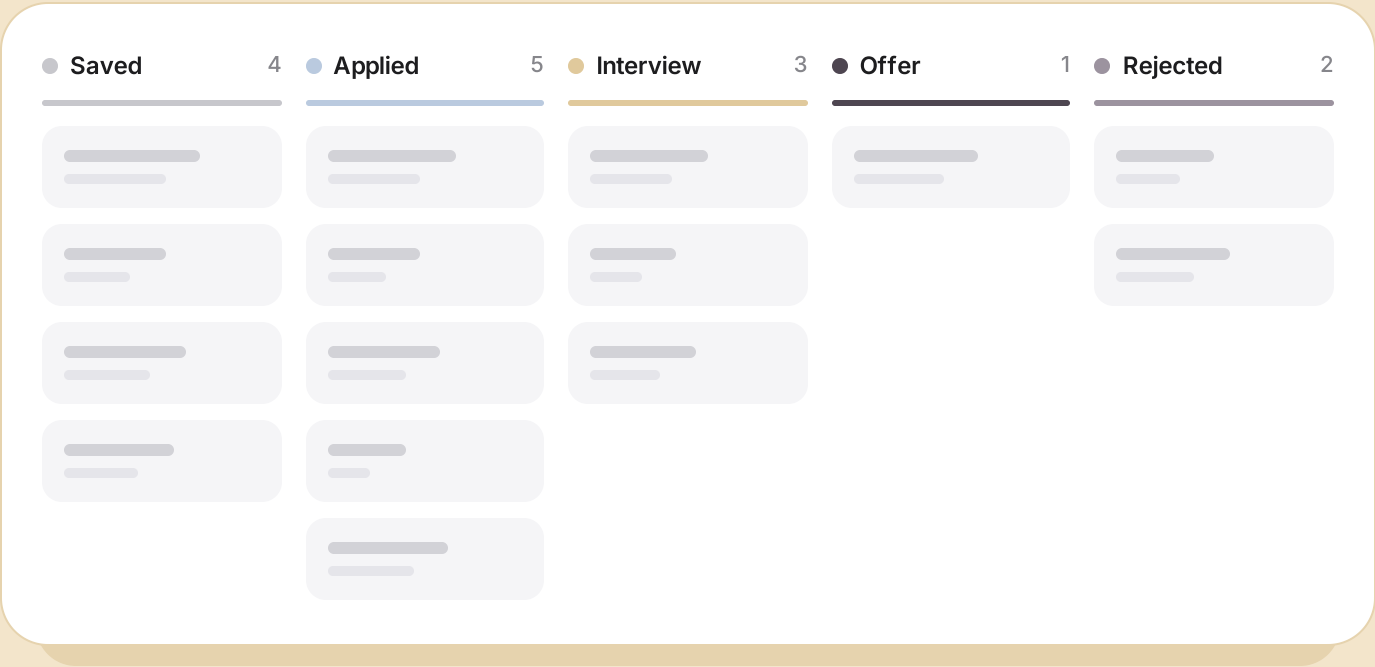


InternTrackAI

An AI-assisted internship application tracker that tells you which roles you actually match, before you apply.



Feature tour	Twelve shipped capabilities
Six diagrams	Journey, architecture, lifecycle, data, AI, deploy
Engineering notes	Three findings, each overturned by measurement
Appendix	Routes and configuration keys

Mjd Arow
Computer Programming & Analysis
George Brown College
interntrackai.majdarow.com
September 2026

What's in this document

● Product ● Architecture ● Intelligence ● Operations ● Security ● Reference

01	At a glance	What it is, who it's for, and the numbers	3
02	The idea	The problem this was built to solve	4
03	Feature tour	Every shipped capability, one card each	5
04	User journey	Sign up through to interview prep	8
05	System architecture	Browser, MVC, services, data, OpenAI	9
06	Request lifecycle	One flow, step by step	10
07	Data model	Entities and relationships	11
08	AI pipeline	How every AI feature is built and guarded	12
09	Field awareness	Why this isn't a software-only tool	13
10	The practice engine	Generation, de-duplication, scoring	14
11	Security and data	Auth, scoping, secrets, deletion	16
12	Deployment	Local to production	17
13	Testing	What's covered, and what isn't	18
14	Engineering notes	Three findings worth reading	19
15	Tech stack	Every choice, and why	21
16	Roadmap	Planned, not built	22
17	Appendix	Routes and configuration keys	23

One place for every application, deadline and interview

InternTrackAI is a full-stack web application that tracks internship and new-grad applications, and uses a language model to do the work around them: reading job descriptions, scoring a resume against a posting, drafting cover letters, and generating interview questions you can answer and be graded on.

12

AI-backed services

1,392

automated tests

13

field categories

1

production region

Who it's for

Students and new graduates running twenty or more applications at once, who lose track of which resume went where, which deadline is next, and which posting they were actually a good fit for.

What makes it different

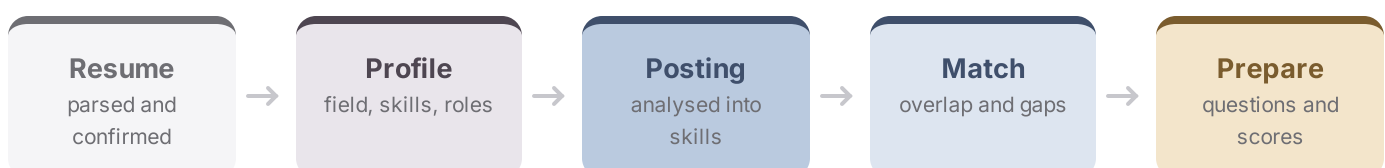
It is not a software-only tool. The app detects the user's field from their resume and adapts every AI prompt to it, so a nursing or accounting student gets questions and feedback from their own discipline rather than a developer's.

Three headline facts

- 1 Deployed to a custom domain and reachable publicly, with a shared demo account that reseeds nightly.
- 2 Every AI call is rate-limited per user across two separate budgets, and capped by a wall-clock timeout.
- 3 Schema correctness is verified against a real PostgreSQL instance, not only the local SQLite database.

How the pieces connect

Each stage produces the input for the next. That chain is what separates this from a spreadsheet with an AI button attached.



The admin around applying is the part nobody builds for

Job boards help you find postings. Spreadsheets help you remember them. Neither helps with the thinking in between.



The tracking problem

Twenty applications across five job boards, three resume versions, and a deadline you only remember because it already passed. A spreadsheet holds the data but answers no questions.



The fit problem

Every posting looks plausible at 11pm. Knowing whether your resume actually covers what a posting asks for requires reading both carefully, side by side, every time.



The preparation problem

Interview prep is generic until it isn't. Practising "tell me about yourself" is not the same as practising for the specific backend role you have an interview for on Thursday.

The premise

Track, analyse, prepare, in one place, where each step feeds the next.

The application you saved becomes the posting the analyser reads, which becomes the resume score, which becomes the cover letter, which becomes the interview questions you practise the night before.

saved application → analysed posting → resume score → cover letter → interview questions

Everything that ships today

Twelve capabilities, all live. Nothing on these three pages is planned or partial.

● Track ● Analyse ● Prepare ● Assist



Application tracking

Track

A list view and a drag-and-drop board across five stages: Saved, Applied, Interview, Offer, Rejected. Deadlines, work mode, notes and per-application history.

Why it matters. Every other feature hangs off an application record.



Dashboard

Track

Counts by stage, applications over time, response rate by resume version, an attention card for what needs doing next, and a practice summary.

Why it matters. Answers "what should I do today" without opening anything else.



Job description analyzer

Analyse

Paste a posting. The model returns the key skills, requirements and responsibilities as structured data, stored against the application.

Why it matters. Turns a wall of text into something the rest of the app can use.



Resume match scoring

Analyse

Scores an uploaded resume against a specific posting and reports the overlap and the gaps as a percentage plus a written explanation.

Why it matters. Tells you whether a posting is worth the hour it takes to apply.

Analyse, draft and prepare



Resume upload and review

Analyse

PDF or Word. Text is extracted server-side, parsed by the model into field, skills with supporting evidence, seniority and target roles, then shown on a review screen where you confirm before anything is written.

Why it matters. No AI output reaches the profile without the user seeing it first.



Cover letter generator

Assist

Drafts a letter from the profile and the posting, with a rewrite pass for tightening specific paragraphs.

Why it matters. The first draft is the hard part; this removes it.



Interview prep

Prepare

Generates questions grounded in a specific posting, split across technical, behavioural and company-specific categories.

Why it matters. Prep for the interview you actually have, not interviews in general.



Practice engine

Prepare

Questions at three difficulties across three categories, with de-duplication, written answers scored one to five with two concrete improvements, retry with side-by-side attempt comparison, and progress tracking.

Why it matters. The part that turns a tracker into a preparation tool.

The work around the application



Inbox suggestions

Track

Optional read-only Gmail connection. Matched messages are classified and surfaced as suggested status changes for you to accept or dismiss.

Why it matters. Rejection emails update the board without you touching it.



Salary insight

Assist

An estimate for a role and location, scoped to the seniority on the profile.

Why it matters. Context before a conversation about numbers.



Follow-up drafting

Assist

Drafts a follow-up message for an application that has gone quiet.

Why it matters. The message you keep meaning to send.



Guided tour

Track

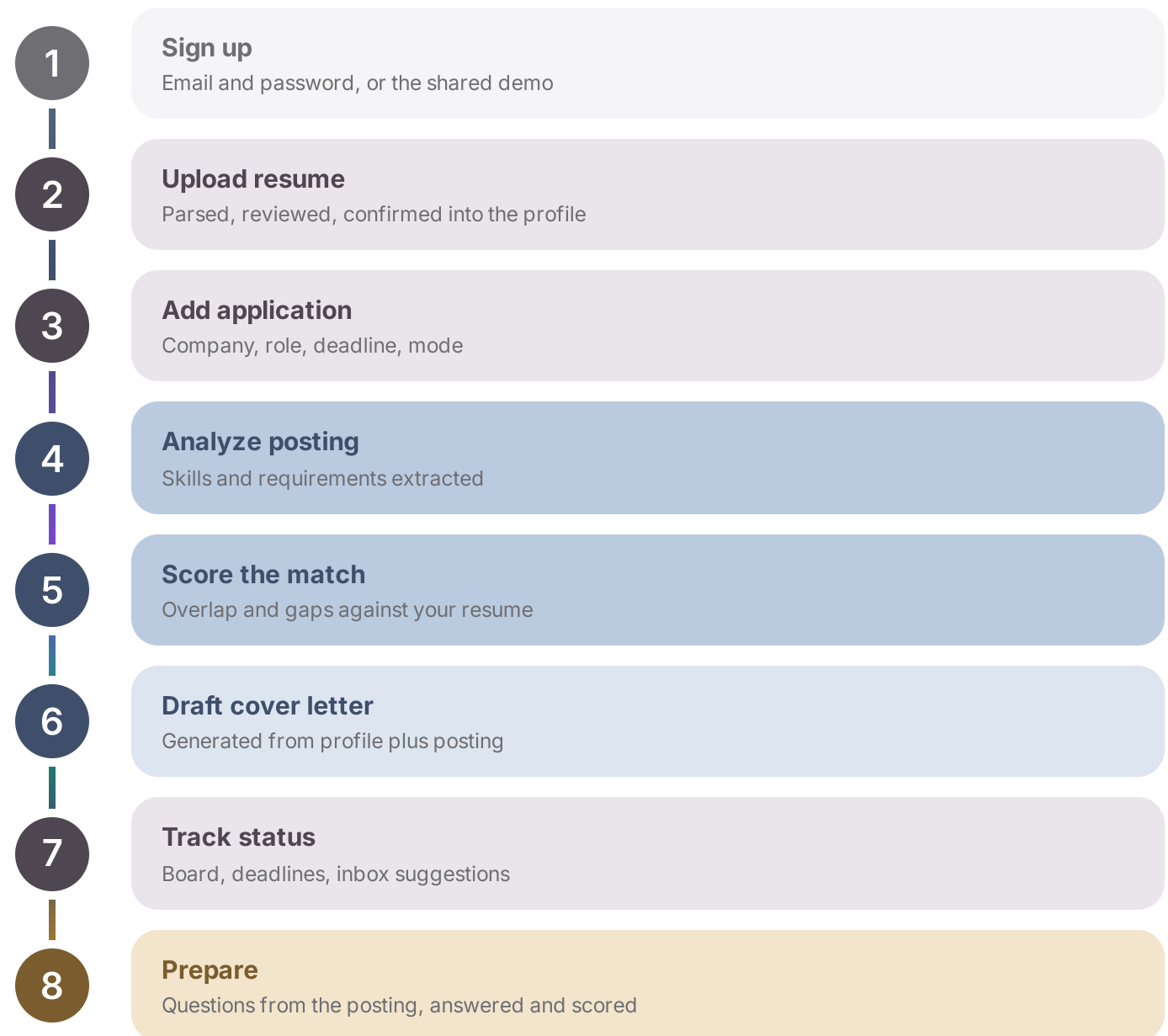
A five-step walkthrough in the order a user actually works: set up, track, prepare. Never auto-plays; offered, not imposed.

Why it matters. A first-time visitor sees the point in sixty seconds.

A note on the number twelve. These twelve user-facing features are not the same list as the twelve AI services on page 12. Tracking, the dashboard and the guided tour call no model at all.

From sign-up to the night before the interview

Each step produces something the next step consumes. That chain is the product.



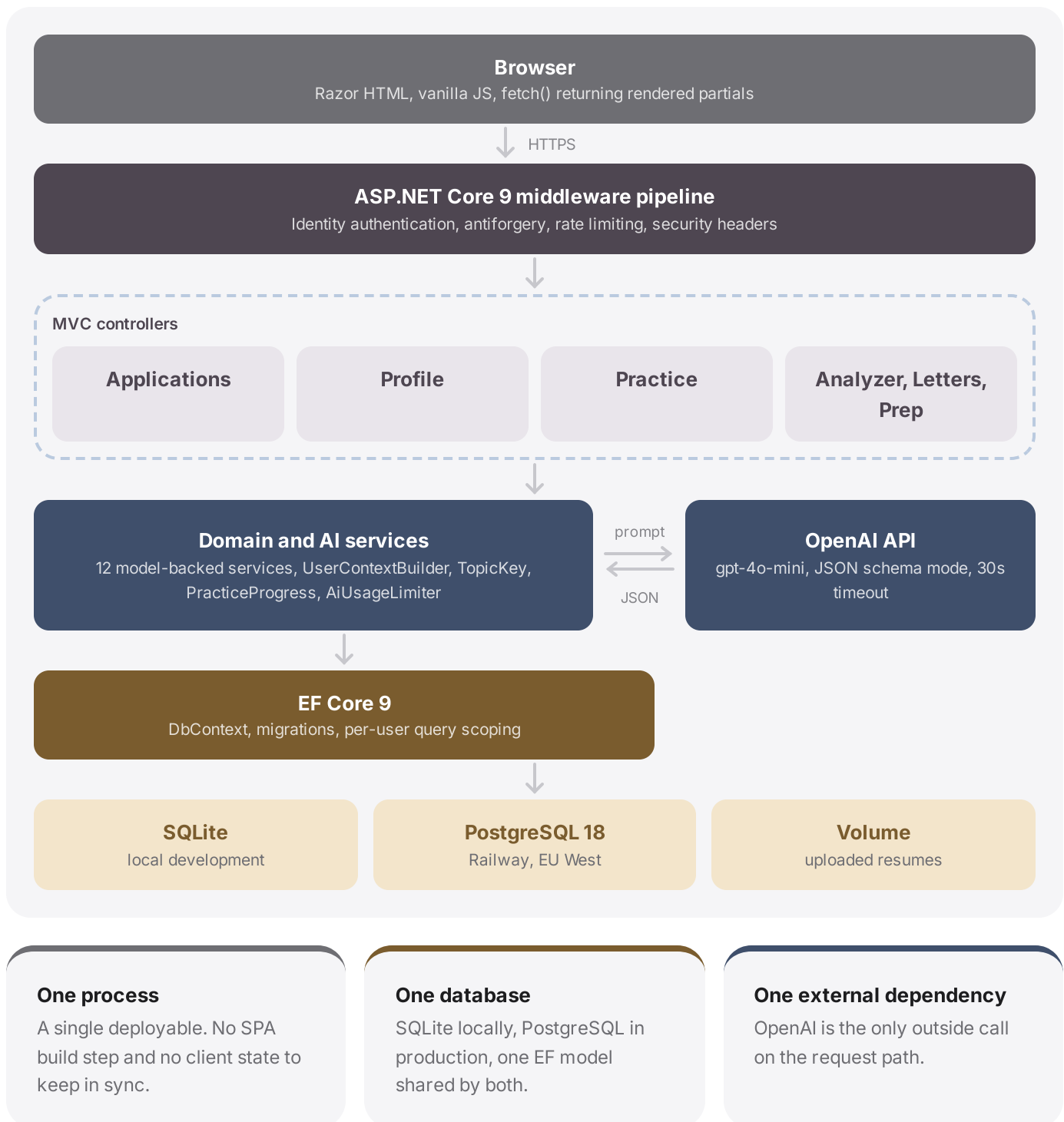
The loop closes

Practice questions generated from an application stay grouped under that application. Answer one and the score appears on both the interview-prep page and the practice page: one question store, three entry points.

Server-rendered, single deployable, no SPA

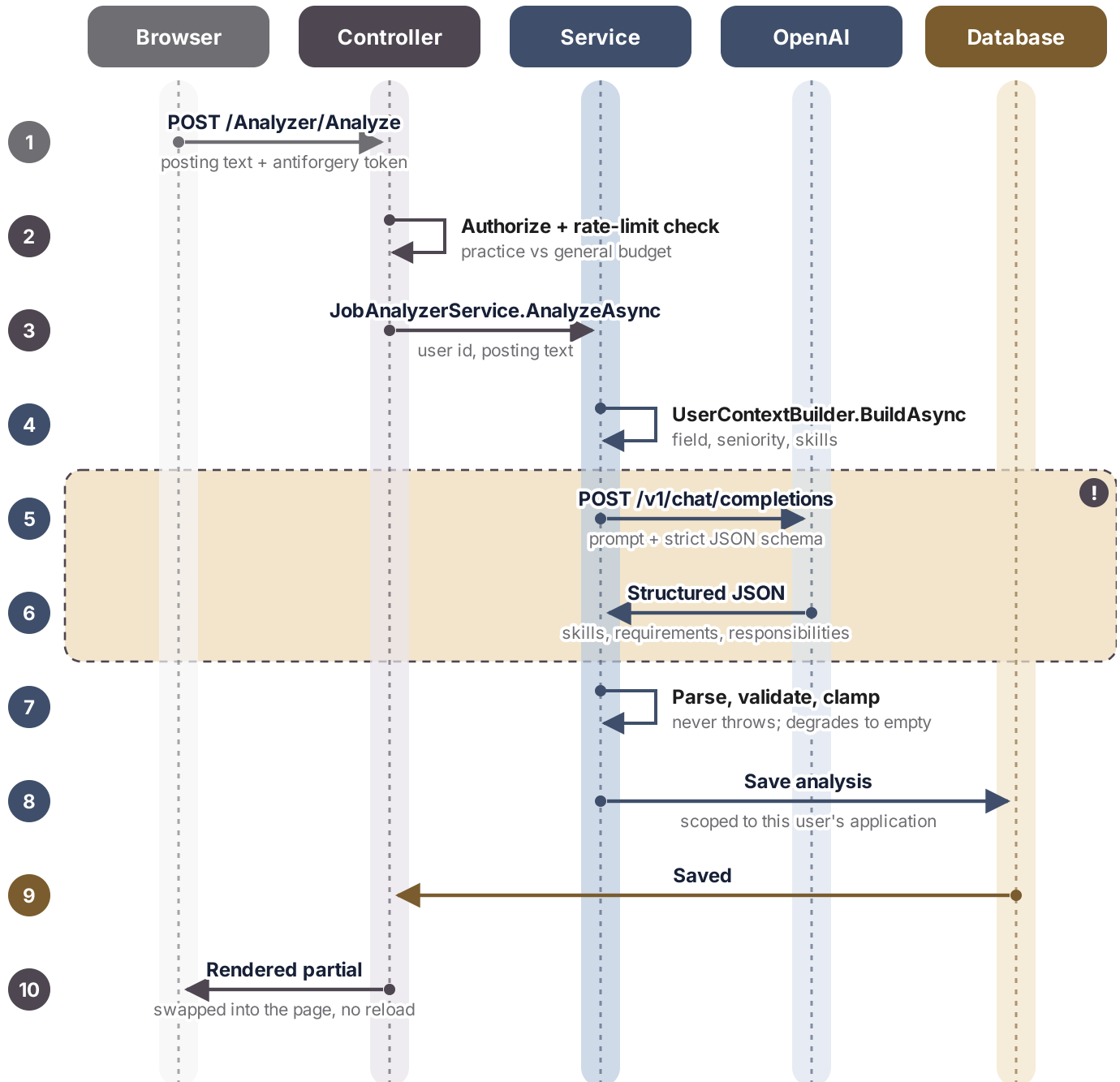
Razor views rendered on the server; dynamic behaviour is vanilla JavaScript fetching rendered partials. One process, one database, one external dependency.

● Client ● In-process ● Services ● Data ● External



One flow: analysing a job description

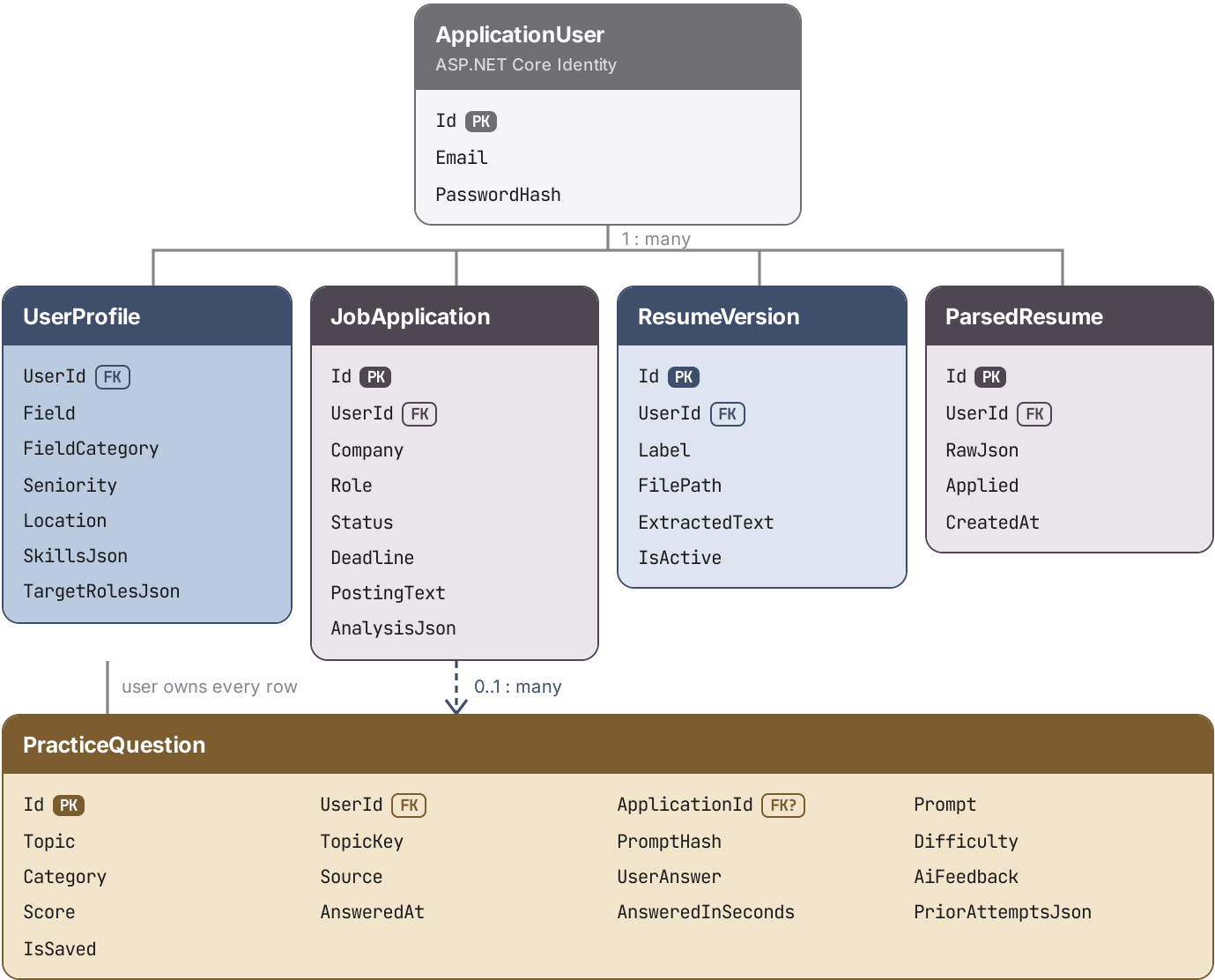
Ten steps from a paste into a textarea to structured skills stored against an application.



Failure at step 5 or 6 never surfaces as an exception. The parser degrades to an empty result, the card shows an error, and the typed input is preserved.

Everything hangs off the user

Every domain table carries a user id, and every query filters on it. Ownership is enforced in the query, not in the view.



UNIQUE (UserId, PromptHash) Two users may be asked the same question; one user cannot be asked it twice.

Enums as integers, explicit values

Appending or reordering a member can never shift stored rows.

Source recorded on every question

Prep and practice questions are distinguishable, which lets topic comparison skip broad ones.

Text columns declared explicitly

An over-long AI summary cannot fail on PostgreSQL while succeeding locally.

Every AI feature is built the same way

Twelve services share one shape. That is deliberate: a single pipeline means a single place to add a guard.



Gather	The user's field, seniority, skills and target roles, plus the feature's own input: a posting, a resume, an answer.
Build	One shared context block is injected into every prompt, so output fits the user's discipline rather than defaulting to software.
Call	gpt-4o-mini through one endpoint helper, with strict JSON schema mode, a token ceiling, and a 30-second timeout.
Parse	Never-throws parsing. Malformed output degrades to a safe empty result rather than raising.
Guard	Domain rules after parsing: enum values validated, scores clamped, empty praise filtered, duplicates rejected.
Persist	Written scoped to the user, and rendered back as a partial the client swaps into the page.

Two budgets, not one

Practice is many small calls; resume parsing and cover letters are rare and expensive. Sharing one budget meant a practice round could lock the resume tools, so they were split.

Per-user limit	Demo account	Signed-in user
Practice	25 per hour	100 per rolling 24h
Everything else	10 per hour	20 per hour

Why a small model

gpt-4o-mini is the only model the app calls. Practice generation and answer scoring are many small calls rather than a few large ones, so per-call cost matters more than peak capability. Strict JSON schema output removes string repair from the parsing path entirely, which is worth more here than a larger model's prose.

max_tokens
on all 12 services

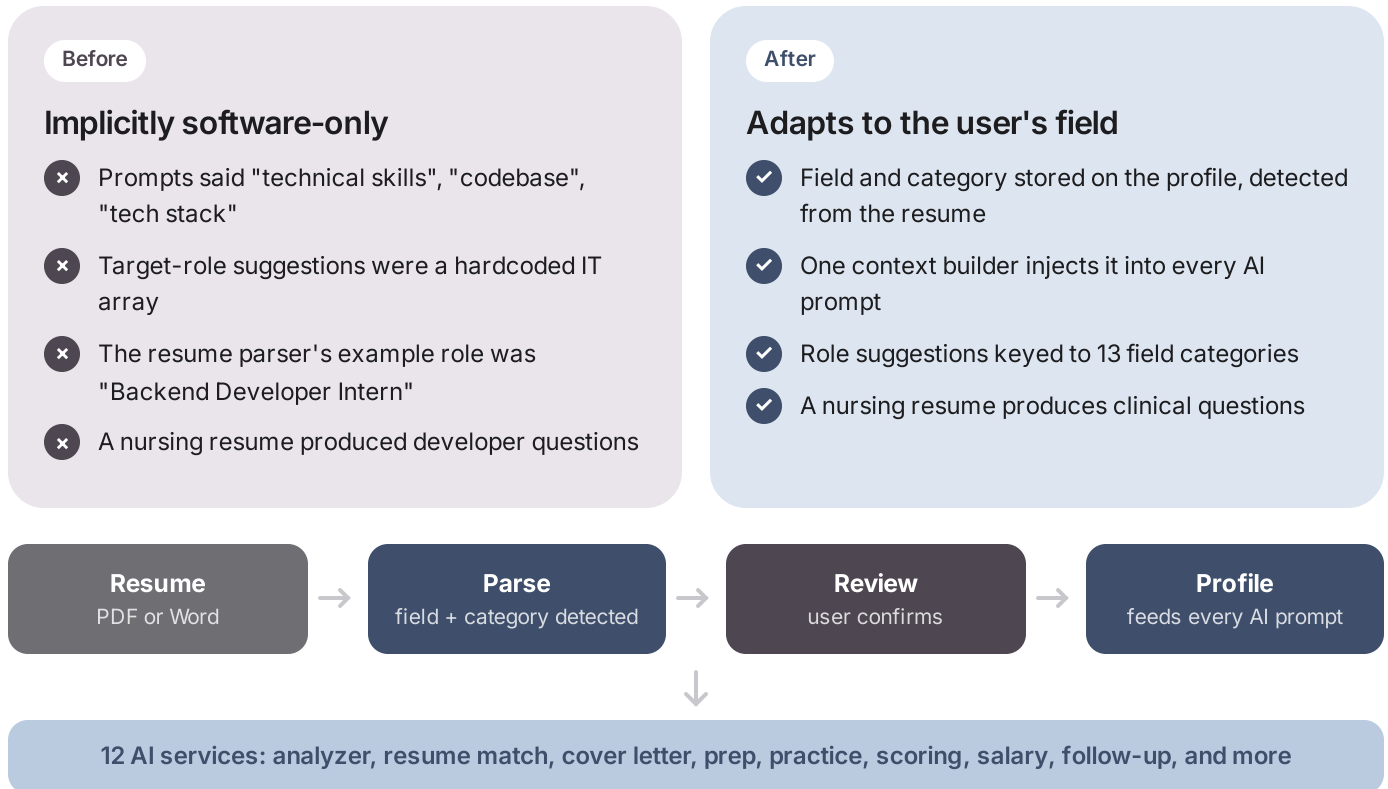
30-second timeout
on all 12 services

Never-throws parsing
degrades to an empty result

One endpoint helper
a test fails if the address is hardcoded

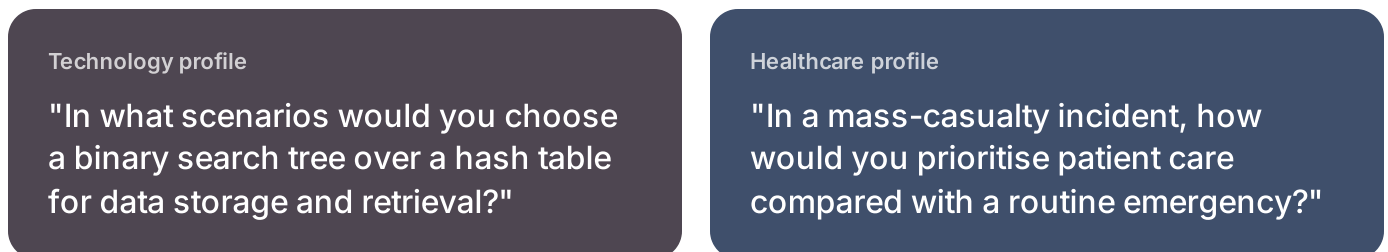
The app is not a software-only tool

The original version assumed every user was a developer. Prompt vocabulary, the role suggestions and the resume parser all had it baked in. A nursing or accounting student got output that didn't fit their discipline.



The same button, two disciplines

Nothing in the code branches on field. One context block changes what comes back.



Both produced by the same endpoint, on the same day, differing only in the profile behind them.

Generating questions is easy. Not repeating them is not.

A model asked not to repeat itself rephrases instead. Measurement showed the prompt layer does nothing; the code layers do the work.

Layer 1

Topic steering

The prompt carries every topic already covered and asks for new ones.

No measurable effect

Layer 2

TopicKey

Normalised topic comparison with subset containment, applied in code.

Caught 4 duplicates in 30

Layer 3

Top-up retry

One extra call for the shortfall when duplicates are dropped. Capped.

Refills what layer 2 drops

A unique (UserId, PromptHash) index sits behind all three as a safety net for exact repeats. In practice it never fired.

What two instrumented live runs measured

Thirty questions generated in one topic area, twice, with every batch logged. The result inverted the design's own assumption.

Prompt exclusion list

no measurable effect on recurrence

0

Hash backstop (string-level)

fired 0 times in 84 questions

0

TopicKey (normalised topics)

caught 4 duplicates in 30

4 / 30

Topic granularity

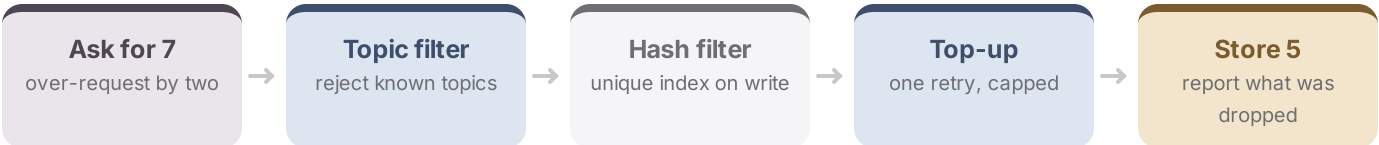
held at every batch; the rule works

held

The finding

The prompt does not steer. The code-level mechanism carries de-duplication almost entirely. Anyone reaching for the prompt to reduce repetition is reaching for the wrong lever.

What one request actually does



Every generation logs what it asked for, what came back, and which layer dropped what.

Answering, scoring and the loop back

1

Answer
40-character minimum, drafts autosaved to the browser and restored on reload, elapsed time recorded.

2

Score
One to five, with what worked, exactly two things to change, what was left out, and a stronger opening line.

3

Retry
The earlier attempt is kept and shown beside the new one, oldest first, so the progression reads left to right.

4

Track
Progress by difficulty, average score, weakest topic, each with a minimum sample size before it will say anything.

The scoring rule that worked, and the one that didn't

Rule A: tighten the prose around grade boundaries

Generic answer

3

Specific answer

4

Rule B: name a condition the model can check

Generic answer

2

Specific answer

4

Rule A changed nothing, so it was removed rather than kept. Rule B asks the model to scan for a number, a date, a named tool, a named role, or a situation that actually happened; if there is none, the answer scores at most 2. The generic answer dropped to 2 while the specific answer held at 4.

Statistics that refuse to speak too early

Every derived figure carries a minimum sample size. A weakest topic needs at least three answers before it is named, the recent-average window needs ten, and a score trend needs twenty. Below those thresholds the panel shows nothing rather than presenting noise as insight.

3

answers before a weakest topic is named

10

answers before the recent-average window appears

20

answers before a trend line is drawn

Scoped by construction, deletable on request

Authentication

ASP.NET Core Identity with cookie authentication. Password reset by emailed token. Optional Google OAuth for the read-only Gmail connection only, never for sign-in.

Secrets

No key, connection string or credential is committed. Local development uses .NET user-secrets; production reads environment variables. The e2e environment cannot read user-secrets at all, so it structurally cannot spend against the real API key.

Gmail scope

One scope is requested: read-only. Message bodies are classified but never stored; only matched metadata and a one-line summary are kept.

Transport and headers

HTTPS end to end behind a proxied CNAME. Security headers are set centrally, with a content security policy in report-only mode pending the removal of the last inline scripts.

Per-user scoping

Every domain query filters on the authenticated user's id. A dedicated test class attempts cross-user access on every write route and asserts a 404 that contains none of the other user's data.

Rate limiting

Two per-user budgets across every AI path, including work started outside a request. Every AI client carries a 30-second timeout and a token ceiling.

Deletion

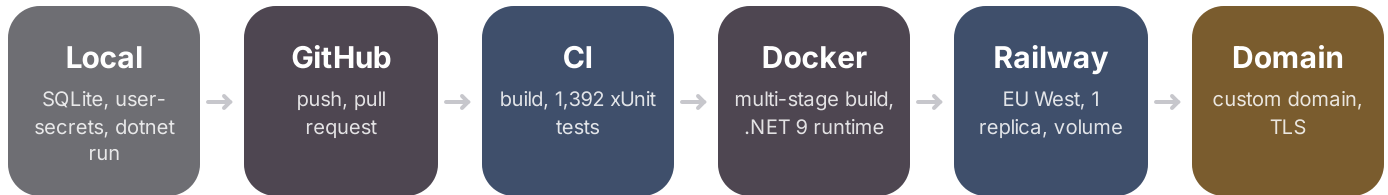
Account deletion revokes the Google grant before wiping data, then clears every user-owned table. A test seeds one row in every table, deletes through the real page, and asserts each table is empty, and fails if a new table was never seeded.

What is never stored

- ✗ Gmail message bodies: classified in memory, never written
- ✗ Any API key, token or connection string in source control
- ✗ Another user's data in any response, including error pages

Local SQLite to production PostgreSQL

One Dockerfile, one Railway service, one database, one region. Every schema change is verified against real PostgreSQL before it ships.



Why one replica

A volume is attached for uploaded resumes, and Railway does not allow replicas on a service with an attached volume. That makes single-instance a structural property rather than a setting, which matters, because the rate limiter holds its buckets in memory per instance.

Two providers, one schema

Migrations must apply cleanly on both SQLite and PostgreSQL. A test reflects over every column and fails on any CLR type it has not seen, so a new type cannot reach production untested. Another asserts no column becomes a length-limited varchar on PostgreSQL.

04:00

UTC nightly reset

The demo account

Publicly reachable, shared, and reset both per session and nightly. Practice data, the resume draft and the profile all return to a fixed seeded state, so one visitor never inherits another's answers.

1,392 tests, and an honest account of the gap

Unit

Pure logic with no I/O: hash normalisation, topic keys, attempt history, progress arithmetic, resume file-type detection, parsed-profile validation.

Integration

In-memory host with a fake model transport. Full request paths: generation, scoring, review and merge, rate limits, grouping, dashboard aggregation.

Ownership

Every write route attempted as the wrong user, asserting a 404 whose body contains none of the other user's seeded data.

Provider parity

The same query run against SQLite and real PostgreSQL, asserting identical results and the absolute expected order, not only that the two agree.

Migration shape

Reflection over every column, failing on an unrecognised CLR type or a length-limited varchar.

Guard tests

Deliberate mutations confirm each test fails when the thing it guards is removed.

What the suite does not cover

Client-side behaviour has no automated coverage. The batch scoring fan-out, draft autosave, collapse state and the tour engine are verified by hand in a real browser. That gap is not theoretical: three bugs this month were invisible to a fully green suite.

- A progress card that never refreshed after generating
- Draft autosave that silently never restored
- A rate-limit message rendering 2,786px above the viewport

Planned: an end-to-end browser suite covering these paths is scoped. It is not built.

Where the tests actually run

Local

SQLite in-memory, fake model transport, about 60 seconds

Real PostgreSQL

migration shape and provider parity, opt-in by connection string

CI

every push and pull request to the default branch

Three findings worth reading

Each of these started as a confident assumption and was overturned by measurement.

01

Production and local disagreed about sorting, for months

Local PostgreSQL was initialised with collation C, which is byte order, the same as SQLite. Production runs en_US.utf8, which is dictionary order. Sorting applications by company name therefore produced a different order in production than on any developer machine, silently, for as long as the feature had existed.

Illustrative, local: collation C
Apple, Zebra, apex, beta

Production: en_US.utf8
apex, Apple, beta, Zebra

The fix was to lower the value in the query, so ordering no longer depends on a database setting the repository does not control. The sharper lesson: a parity test asserting only that two providers agree would have passed here and proved nothing, because both agreed on the wrong answer. The test now asserts the absolute expected order.

02

The prompt was not the lever

Question de-duplication was designed with prompt steering as the primary mechanism and code-level hashing as a backstop. Two instrumented live runs measured the opposite: the hash never fired across 84 questions, while normalised topic comparison caught four duplicates in thirty.

A second measurement refined it further. A prompt asking the model to make a judgement does not land; a prompt naming a condition the model can check does.

Changed nothing
"A generic answer is a 2."

Moved the score as intended
"Scan for a number, a date, a named tool, or a situation that happened. If there is none, at most 2."

The third finding, and the pattern

03

A safety net was hiding the thing it sat next to

A draft-autosave feature silently never restored. A try/catch written to tolerate private-mode browsers, where local storage throws, was broad enough to also swallow a `ReferenceError` from an unrelated declaration-order bug in the same file. Every server-side test passed throughout.

A sweep of the codebase found fifteen more multi-statement catch blocks, including one wrapping thirty-nine statements that answered every possible failure with "check your connection", so a rendering bug would have sent a user to their network settings.

`ReferenceError`: re-thrown

`TypeError`: passed through

`SyntaxError`: passed through

A shared guard now re-throws `ReferenceError`, which is always a programming error, while still passing through `TypeError` and `SyntaxError`, the two ways `fetch` and `JSON.parse` legitimately report the conditions those catches were written for. A blanket re-throw would have traded silent failures for noisy false alarms.

The pattern underneath all three

In each case, something that looked like proof wasn't. A check passed, or a design looked sound, while the thing it was meant to observe was broken or idle.

Parity test

Both providers agreed, on the wrong answer.

De-dup design

The mechanism meant to do the work never fired; a different one did.

Catch block

Swallowed the very error it sat next to.

The habit that caught them was asking what a green result, or a confident design, is structurally incapable of proving, and then measuring.

Every choice, and why

Technology	Role	Why
● ASP.NET Core 9 MVC	Server-rendered Razor	Server rendering keeps one language and one deployable. No SPA build step, no client state to keep in sync.
● EF Core 9	ORM and migrations	Migrations as code, and one model definition that both providers share.
● PostgreSQL 18	Production database	Managed on Railway, with backups and a real type system worth targeting.
● SQLite	Local development	Zero-setup local runs and a fast in-memory database for the test suite.
● OpenAI gpt-4o-mini	Language model	Cheap enough for many small calls, with strict JSON schema output that removes string repair from the parsing path.
● ASP.NET Core Identity	Authentication	Password hashing, token handling and cookie management that should not be hand-rolled.
● xUnit	Testing	Integration tests run the real request pipeline against an in-memory host.
● Docker	Packaging	A multi-stage build so the runtime image carries no SDK.
● Railway	Hosting	Deploys from a Dockerfile, provisions PostgreSQL, and handles TLS on a custom domain.
● Cloudflare	DNS	Proxied CNAME to the Railway service.
● Resend	Transactional email	Password reset only. No marketing, no lists.
● Vanilla JavaScript	Client behaviour	Partials rendered on the server and swapped in. No framework, because nothing on the page needs one.

Planned, not built

Everything on this page is scoped and deliberately deferred. None of it exists in the codebase today.



End-to-end browser suite

Planned

Practice and tour dimensions driving a real browser, closing the client-side coverage gap named on page 18. Scoped, with the specific bugs each check would have caught already written down.



Content security policy enforcement

Planned

Five inline scripts remain, all in two layout files. Moving them to files allows the policy to move from report-only to enforced.



Generator consolidation

Planned

Interview prep and practice currently run two generators against one question store. Folding them into one removes a second prompt to maintain.



Practice history view

Planned

Deliberately deferred until there is enough data for it to say anything: forty or more answered questions, three topics with at least three answers each, and retries becoming common.

Decided against

A pre-generated question seed bank

Scoped and rejected: roughly 936 model calls for 4,680 questions that could not be quality-checked, and global rows that would fight a deliberately non-nullable user id. Documented as a decision, not a backlog item.

Routes and configuration

Principal routes

GET	/	Landing page with live preview
GET	/Home/Dashboard	Stats, attention, inbox, practice summary
GET	/JobApplications	List and board views
POST	/JobApplications/Create	Add an application
POST	/Analyzer/Analyze	Extract skills from a posting
POST	/Profile/UploadResume	Upload and parse a resume
GET	/Profile/ReviewResume	Confirm parsed data before merge
POST	/Profile/ApplyResumeReview	Merge confirmed data into profile
POST	/Profile/ScoreResume	Score resume against a posting
POST	/CoverLetter/Generate	Draft a cover letter
GET	/InterviewPrep/Prep	Posting-grounded questions
POST	/InterviewPrep/Generate	Generate more prep questions
GET	/Practice	Practice page with filters and progress
POST	/Practice/GenerateMore	Generate questions
POST	/Practice/SubmitAnswer	Score an answer
GET	/Practice/Progress	Refreshed progress partial
POST	/Practice/DeleteAll	Delete all practice data
GET	/Home/Privacy	Privacy policy
GET	/Home/Terms	Terms
GET	/health	Health check for the platform

Configuration keys (names only; no values appear in this document)

ASPNETCORE_ENVIRONMENT DATABASE_URL UPLOADS_PATH OpenAI__ApiKey OpenAI__BaseUrl Admin__Email
 Demo__Email Demo__Password Demo__AutoReset Email__From Email__BaseUrl Resend__ApiKey
 Google__ClientId Google__ClientSecret RateLimiting__AI__Practice__PermitLimit

No key, connection string, token or real user record appears anywhere in this document. Local development reads .NET user-secrets; production reads environment variables set on the platform.